Practical: Population-Adjusted Indirect Comparisons with outstandR

Department of Statistical Science, University College London (UCL)

Introduction and Applying PAIC Methods

Background

When head-to-head randomized controlled trials (RCTs) are unavailable, Health Technology Assessment (HTA) relies on indirect comparisons. We frequently face a scenario where we have **Individual Patient Data (IPD)** from a trial comparing our treatment A to a common comparator C (the AC trial), and **Aggregate Level Data (ALD)** from a published trial comparing treatment B to the same comparator C (the BC trial).

If these trial populations differ in **effect modifiers** (patient characteristics that change the magnitude of the treatment effect), naive indirect comparisons will be biased. **Population adjustment methods** correct for these differences, simulating how treatment A would have performed if it had been tested in the BC trial's population. We will use the `{outstandR}` package to structure IPD and ALD for analysis, implement explicit two-part formulas, and apply various standardisation methods.

Exercises

1. First, load the necessary `R` packages and set a seed for reproducibility. Ensure `{outstandR}` is installed.

```
library(outstandR)
library(dplyr)
library(ggplot2)
```

2. Load the built-in binary outcome datasets and inspect their structure. The IPD is in standard wide format, while the ALD is in a "tidy" long format.

```r
data(AC_IPD_binY_contX) # IPD for treatments A vs C
data(BC_ALD_binY_contX) # ALD for treatments B vs C

head(AC_IPD_binY_contX)
```

```
  id PF_cont_1   PF_cont_2   EM_cont_1   EM_cont_2 trt y    true_eta
1  1 0.0826453  0.71645068  0.66422255 -0.09202652   A 0 -1.58891404
2  2 0.6957433  0.38608723  0.79875530  0.74880584   C 1  0.14986776
3  3 0.6516889  0.50572444  0.39944706 -0.07230380   A 0 -1.43868552
4  4 0.2165975  0.35783123  0.25957984 -0.55405590   A 0 -2.09172428
5  5 0.2605580 -0.05826527 -0.10696235  0.33762456   A 0 -2.13936317
6  6 0.1110818  0.64041589  0.04867791  0.34980415   C 1 -0.07910151
```

```r
print(BC_ALD_binY_contX)
```

```
# A tibble: 16 x 4
   variable  statistic   value trt
   <chr>     <chr>       <dbl> <chr>
 1 EM_cont_1 mean        0.651 <NA>
 2 EM_cont_1 sd          0.391 <NA>
 3 EM_cont_2 mean        0.592 <NA>
 4 EM_cont_2 sd          0.416 <NA>
 5 PF_cont_1 mean        0.653 <NA>
 6 PF_cont_1 sd          0.371 <NA>
 7 PF_cont_2 mean        0.583 <NA>
 8 PF_cont_2 sd          0.437 <NA>
 9 y         mean        0.615 C
10 y         sd          0.490 C
11 y         sum        40     C
12 y         mean        0.274 B
13 y         sd          0.448 B
14 y         sum        37     B
15 <NA>      N          65     C
16 <NA>      N         135     B
```

3. Define the model formulas. `{outstandR}` uses an explicit two-part list for formulas to separate the intent of regression modeling from weighting.

```r
# 1. Outcome Regression Model (used for G-computation)
lin_form_bin <- as.formula(
  "y ~ PF_cont_1 + PF_cont_2 + trt + trt:EM_cont_1 + trt:EM_cont_2")

# 2. Balance Model (used for MAIC weighting)
bal_form_bin <- as.formula("~ EM_cont_1 + EM_cont_2")

formula_list_bin <- list(outcome_model = lin_form_bin,
                         balance_model = bal_form_bin)
```

4. Firstly apply Matching-Adjusted Indirect Comparison (MAIC). This utilizes a method-of-moments approach, estimating weights for the IPD patients so their aggregate baseline characteristics match the target ALD trial.

The `outstandR()` function is the primary high-level wrapper in the `outstandR` package that orchestrates the entire population-adjusted indirect comparison (PAIC) standardization process. It is designed to abstract away the complex task of transporting treatment effects from the individual patient data (IPD) population to the aggregate-level data (ALD) population. By remaining agnostic to the specific adjustment method being used, it provides a unified interface where computational heavy lifting is delegated to the appropriate backend via an S3 class dispatch system.

Function Arguments

The function accepts a variety of arguments to define the data, the adjustment strategy, and the desired outputs:

- **`ipd_trial`**: Individual-level patient data (e.g., for treatments A and C), expected in a long format including treatment and outcome columns consistent with the formula object.
- **`ald_trial`**: Aggregate-level data (e.g., for treatments B and C). This should contain specific columns for `variable` (covariate name), `statistic` (e.g., mean, sd, prop, sum), `value` (numerical summary statistic), and `trt` (treatment label, or NA if a common covariate distribution is assumed).
- **`strategy`**: A computation strategy function determining the adjustment method. Options include `strategy_maic()`, `strategy_stc()`, `strategy_gcomp_ml()`, `strategy_gcomp_bayes()`, or `strategy_mim()`.
- **`ref_trt`**: The reference, common, or anchoring treatment name (can also be referred to as the index, while the comparator is called the reference).
- **`CI`**: The desired confidence interval level (between **0** and **1**, defaulting to **0.95**).

- **scale**: The relative treatment effect scale. If NULL, it is automatically determined from the model. Options include log_odds, log_relative_risk, risk_difference, mean_difference, or rate_difference, depending on the data type.
- **var_method**: The method of variance estimation. Options are sample (default), sandwich, and pool.
- **verbose**: Logical (default TRUE). Controls console output and provides proactive warnings for computationally expensive operations to prevent users from assuming the session has hung.
- **seed**: Random number seed (optional).
- ...: Additional arguments, allowing named arguments to be passed to functions like rstanarm::stan_glm() via strategy_gcomp_bayes().

```
out_maic <-
 outstandR(
   ipd_trial = AC_IPD_binY_contX,
   ald_trial = BC_ALD_binY_contX,
   strategy = strategy_maic(
     formula = list(outcome_model = formula("y ~ trt"),
                    balance_model =  bal_form_bin),
     family = binomial(link = "logit")),
   seed = 12345)

print(out_maic)
```

```
Object of class 'outstandR'
ITC algorithm: MAIC
Model: binomial
Scale: log_odds
Common treatment: C
Individual patient data study: A vs C
Aggregate level data study: B vs C
Confidence interval level: 0.95

Contrasts:

# A tibble: 3 x 5
  Treatments Estimate Std.Error lower.0.95 upper.0.95
  <chr>         <dbl>     <dbl>      <dbl>      <dbl>
1 AB            0.620     0.254     -0.368     1.61
2 AC           -0.824     0.152     -1.59     -0.0607
3 BC           -1.44      0.102     -2.07     -0.817
```

```
Absolute:

# A tibble: 3 x 5
  Treatments Estimate Std.Error lower.0.95 upper.0.95
  <chr>         <dbl>     <dbl>      <dbl>      <dbl>
1 A             0.241   0.00208      0.151      0.330
2 B            -1.03    0.108       -1.67      -0.384
3 C             0.417   0.00534      0.274      0.561
```

Apply other models.

```r
out_gcomp_ml <- outstandR(
  ipd_trial = AC_IPD_binY_contX,
  ald_trial = BC_ALD_binY_contX,
  strategy = strategy_gcomp_ml(
    formula = formula_list_bin,
    family = binomial(link = "logit")
  ),
  seed = 12345
)

out_gcomp_bayes <- outstandR(
  ipd_trial = AC_IPD_binY_contX,
  ald_trial = BC_ALD_binY_contX,
  strategy = strategy_gcomp_bayes(
    formula = formula_list_bin,
    family = binomial(link = "logit")
  ),
  seed = 12345
)
```

```
SAMPLING FOR MODEL 'bernoulli' NOW (CHAIN 1).
Chain 1:
Chain 1: Gradient evaluation took 5.7e-05 seconds
Chain 1: 1000 transitions using 10 leapfrog steps per transition would take 0.57 seconds.
Chain 1: Adjust your expectations accordingly!
Chain 1:
Chain 1:
Chain 1: Iteration:    1 / 2000 [  0%]  (Warmup)
Chain 1: Iteration:  500 / 2000 [ 25%]  (Warmup)
Chain 1: Iteration: 1000 / 2000 [ 50%]  (Warmup)
```

```
Chain 1: Iteration: 1001 / 2000 [ 50%]  (Sampling)
Chain 1: Iteration: 1500 / 2000 [ 75%]  (Sampling)
Chain 1: Iteration: 2000 / 2000 [100%]  (Sampling)
Chain 1:
Chain 1:  Elapsed Time: 0.224 seconds (Warm-up)
Chain 1:                0.204 seconds (Sampling)
Chain 1:                0.428 seconds (Total)
Chain 1:

SAMPLING FOR MODEL 'bernoulli' NOW (CHAIN 2).
Chain 2:
Chain 2: Gradient evaluation took 1.7e-05 seconds
Chain 2: 1000 transitions using 10 leapfrog steps per transition would take 0.17 seconds.
Chain 2: Adjust your expectations accordingly!
Chain 2:
Chain 2:
Chain 2: Iteration:    1 / 2000 [  0%]  (Warmup)
Chain 2: Iteration:  500 / 2000 [ 25%]  (Warmup)
Chain 2: Iteration: 1000 / 2000 [ 50%]  (Warmup)
Chain 2: Iteration: 1001 / 2000 [ 50%]  (Sampling)
Chain 2: Iteration: 1500 / 2000 [ 75%]  (Sampling)
Chain 2: Iteration: 2000 / 2000 [100%]  (Sampling)
Chain 2:
Chain 2:  Elapsed Time: 0.216 seconds (Warm-up)
Chain 2:                0.2 seconds (Sampling)
Chain 2:                0.416 seconds (Total)
Chain 2:
```

```r
out_mim <- outstandR(
  ipd_trial = AC_IPD_binY_contX,
  ald_trial = BC_ALD_binY_contX,
  strategy = strategy_mim(
    formula = formula_list_bin,
    family = binomial(link = "logit")
  ),
  seed = 12345
)
```

```
SAMPLING FOR MODEL 'bernoulli' NOW (CHAIN 1).
Chain 1:
Chain 1: Gradient evaluation took 2.5e-05 seconds
```

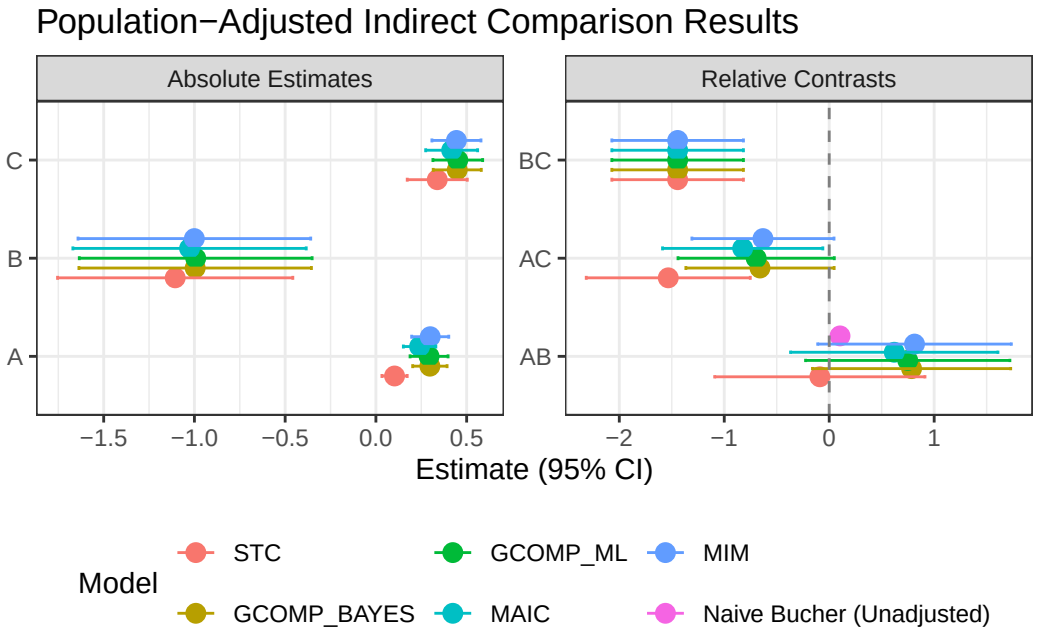
```
Chain 1: 1000 transitions using 10 leapfrog steps per transition would take 0.25 seconds.
Chain 1: Adjust your expectations accordingly!
Chain 1:
Chain 1:
Chain 1: Iteration:    1 / 2000 [  0%]  (Warmup)
Chain 1: Iteration:  200 / 2000 [ 10%]  (Warmup)
Chain 1: Iteration:  400 / 2000 [ 20%]  (Warmup)
Chain 1: Iteration:  600 / 2000 [ 30%]  (Warmup)
Chain 1: Iteration:  800 / 2000 [ 40%]  (Warmup)
Chain 1: Iteration: 1000 / 2000 [ 50%]  (Warmup)
Chain 1: Iteration: 1001 / 2000 [ 50%]  (Sampling)
Chain 1: Iteration: 1200 / 2000 [ 60%]  (Sampling)
Chain 1: Iteration: 1400 / 2000 [ 70%]  (Sampling)
Chain 1: Iteration: 1600 / 2000 [ 80%]  (Sampling)
Chain 1: Iteration: 1800 / 2000 [ 90%]  (Sampling)
Chain 1: Iteration: 2000 / 2000 [100%]  (Sampling)
Chain 1:
Chain 1:  Elapsed Time: 0.224 seconds (Warm-up)
Chain 1:                0.209 seconds (Sampling)
Chain 1:                0.433 seconds (Total)
Chain 1:

SAMPLING FOR MODEL 'bernoulli' NOW (CHAIN 2).
Chain 2:
Chain 2: Gradient evaluation took 1.7e-05 seconds
Chain 2: 1000 transitions using 10 leapfrog steps per transition would take 0.17 seconds.
Chain 2: Adjust your expectations accordingly!
Chain 2:
Chain 2:
Chain 2: Iteration:    1 / 2000 [  0%]  (Warmup)
Chain 2: Iteration:  200 / 2000 [ 10%]  (Warmup)
Chain 2: Iteration:  400 / 2000 [ 20%]  (Warmup)
Chain 2: Iteration:  600 / 2000 [ 30%]  (Warmup)
Chain 2: Iteration:  800 / 2000 [ 40%]  (Warmup)
Chain 2: Iteration: 1000 / 2000 [ 50%]  (Warmup)
Chain 2: Iteration: 1001 / 2000 [ 50%]  (Sampling)
Chain 2: Iteration: 1200 / 2000 [ 60%]  (Sampling)
Chain 2: Iteration: 1400 / 2000 [ 70%]  (Sampling)
Chain 2: Iteration: 1600 / 2000 [ 80%]  (Sampling)
Chain 2: Iteration: 1800 / 2000 [ 90%]  (Sampling)
Chain 2: Iteration: 2000 / 2000 [100%]  (Sampling)
Chain 2:
Chain 2:  Elapsed Time: 0.208 seconds (Warm-up)
```

```
Chain 2:                0.216 seconds (Sampling)
Chain 2:                0.424 seconds (Total)
Chain 2:
```

```r
out_stc <- outstandR(
  ipd_trial = AC_IPD_binY_contX,
  ald_trial = BC_ALD_binY_contX,
  strategy = strategy_stc(
    formula = formula_list_bin,
    family = binomial(link = "logit")
  ),
  seed = 12345
)
```

{outstandR} has an inbuilt plotting function to combine the different outputs into a single forest plot comparing all results.

```r
plot(out_stc, out_gcomp_bayes, out_gcomp_ml, out_maic, out_mim)
```



Note: By default, a binomial model with a logit link returns contrasts on the log-odds scale.

5. Health economic models often require inputs on absolute scales. Run the MAIC again, changing the scale and variance estimator.

> **Tip**
>
> **Task:** Output the results as a Risk Difference using `scale = "risk_difference"`, and use the fast robust sandwich estimator for variance using `var_method = "sandwich"`. The package will automatically estimate the baseline risk from the reference treatment arm to convert the log-odds back to probabilities.

```
out_maic_rd <- outstandR(
  ipd_trial = AC_IPD_binY_contX,
  ald_trial = BC_ALD_binY_contX,
  strategy = strategy_maic(
    formula = list(outcome_model = formula("y ~ trt"),
                   balance_model =  bal_form_bin),
    family = binomial(link = "logit")),
  scale = "risk_difference",
  var_method = "sandwich",
  seed = 12345
)

print(out_maic_rd)
```

```
Object of class 'outstandR'
ITC algorithm: MAIC
Model: binomial
Scale: risk_difference
Common treatment: C
Individual patient data study: A vs C
Aggregate level data study: B vs C
Confidence interval level: 0.95

Contrasts:

# A tibble: 3 x 5
  Treatments Estimate Std.Error lower.0.95 upper.0.95
  <chr>         <dbl>     <dbl>      <dbl>      <dbl>
1 AB            0.165   0.441       -1.14      1.47
2 AC           -0.177   0.00499     -0.315    -0.0381
3 BC           -0.341   0.436       -1.63      0.952

Absolute:

# A tibble: 3 x 5
```

```
  Treatments Estimate Std.Error lower.0.95 upper.0.95
  <chr>         <dbl>     <dbl>      <dbl>      <dbl>
1 A             0.241   0.00208      0.151      0.330
2 B             0.0760  0.441       -1.23       1.38
3 C             0.417   0.00534      0.274      0.561
```

6. Run other methods. We will also extract this on the `risk_difference` scale so we can directly compare it to our MAIC estimate.

```
out_gcomp_ml_rd <- outstandR(
  ipd_trial = AC_IPD_binY_contX,
  ald_trial = BC_ALD_binY_contX,
  strategy = strategy_gcomp_ml(
    formula = formula_list_bin,
    family = binomial(link = "logit")
  ),
  scale = "risk_difference",
  seed = 12345
)

print(out_gcomp_ml_rd)
```

```
Object of class 'outstandR'
ITC algorithm: GCOMP_ML
Model: binomial
Scale: risk_difference
Common treatment: C
Individual patient data study: A vs C
Aggregate level data study: B vs C
Confidence interval level: 0.95

Contrasts:

# A tibble: 3 x 5
  Treatments Estimate Std.Error lower.0.95 upper.0.95
  <chr>         <dbl>     <dbl>      <dbl>      <dbl>
1 AB            0.183   0.443       -1.12      1.49
2 AC           -0.158   0.00728     -0.326     0.00900
3 BC           -0.341   0.436       -1.63      0.952

Absolute:
```

```
# A tibble: 3 x 5
  Treatments Estimate Std.Error lower.0.95 upper.0.95
  <chr>         <dbl>     <dbl>      <dbl>      <dbl>
1 A             0.293   0.00285      0.188      0.398
2 B             0.110   0.441       -1.19       1.41
3 C             0.451   0.00489      0.314      0.588
```

```r
out_gcomp_bayes_rd <- outstandR(
  ipd_trial = AC_IPD_binY_contX,
  ald_trial = BC_ALD_binY_contX,
  strategy = strategy_gcomp_bayes(
    formula = formula_list_bin,
    family = binomial(link = "logit")
  ),
  scale = "risk_difference",
  seed = 12345
)
```

```
SAMPLING FOR MODEL 'bernoulli' NOW (CHAIN 1).
Chain 1:
Chain 1: Gradient evaluation took 3e-05 seconds
Chain 1: 1000 transitions using 10 leapfrog steps per transition would take 0.3 seconds.
Chain 1: Adjust your expectations accordingly!
Chain 1:
Chain 1:
Chain 1: Iteration:    1 / 2000 [  0%]  (Warmup)
Chain 1: Iteration:  500 / 2000 [ 25%]  (Warmup)
Chain 1: Iteration: 1000 / 2000 [ 50%]  (Warmup)
Chain 1: Iteration: 1001 / 2000 [ 50%]  (Sampling)
Chain 1: Iteration: 1500 / 2000 [ 75%]  (Sampling)
Chain 1: Iteration: 2000 / 2000 [100%]  (Sampling)
Chain 1:
Chain 1:  Elapsed Time: 0.225 seconds (Warm-up)
Chain 1:                0.206 seconds (Sampling)
Chain 1:                0.431 seconds (Total)
Chain 1:

SAMPLING FOR MODEL 'bernoulli' NOW (CHAIN 2).
Chain 2:
Chain 2: Gradient evaluation took 1.7e-05 seconds
Chain 2: 1000 transitions using 10 leapfrog steps per transition would take 0.17 seconds.
```

```
Chain 2: Adjust your expectations accordingly!
Chain 2:
Chain 2:
Chain 2: Iteration:    1 / 2000 [  0%]  (Warmup)
Chain 2: Iteration:  500 / 2000 [ 25%]  (Warmup)
Chain 2: Iteration: 1000 / 2000 [ 50%]  (Warmup)
Chain 2: Iteration: 1001 / 2000 [ 50%]  (Sampling)
Chain 2: Iteration: 1500 / 2000 [ 75%]  (Sampling)
Chain 2: Iteration: 2000 / 2000 [100%]  (Sampling)
Chain 2:
Chain 2:  Elapsed Time: 0.214 seconds (Warm-up)
Chain 2:                0.201 seconds (Sampling)
Chain 2:                0.415 seconds (Total)
Chain 2:
```

```
print(out_gcomp_bayes_rd)
```

```
Object of class 'outstandR'
ITC algorithm: GCOMP_BAYES
Model: binomial
Scale: risk_difference
Common treatment: C
Individual patient data study: A vs C
Aggregate level data study: B vs C
Confidence interval level: 0.95

Contrasts:

# A tibble: 3 x 5
  Treatments Estimate Std.Error lower.0.95 upper.0.95
  <chr>         <dbl>     <dbl>      <dbl>      <dbl>
1 AB            0.191   0.442       -1.11     1.49
2 AC           -0.151   0.00670     -0.311    0.00965
3 BC           -0.341   0.436       -1.63     0.952

Absolute:

# A tibble: 3 x 5
  Treatments Estimate Std.Error lower.0.95 upper.0.95
  <chr>         <dbl>     <dbl>      <dbl>      <dbl>
1 A             0.297   0.00234      0.203      0.392
2 B             0.107   0.440       -1.19       1.41
```

```
3 C             0.448   0.00457      0.316      0.581
```

```r
out_mim_rd <- outstandR(
  ipd_trial = AC_IPD_binY_contX,
  ald_trial = BC_ALD_binY_contX,
  strategy = strategy_mim(
    formula = formula_list_bin,
    family = binomial(link = "logit")
  ),
  scale = "risk_difference",
  seed = 12345
)
```

```
SAMPLING FOR MODEL 'bernoulli' NOW (CHAIN 1).
Chain 1:
Chain 1: Gradient evaluation took 2.4e-05 seconds
Chain 1: 1000 transitions using 10 leapfrog steps per transition would take 0.24 seconds.
Chain 1: Adjust your expectations accordingly!
Chain 1:
Chain 1:
Chain 1: Iteration:    1 / 2000 [  0%]  (Warmup)
Chain 1: Iteration:  200 / 2000 [ 10%]  (Warmup)
Chain 1: Iteration:  400 / 2000 [ 20%]  (Warmup)
Chain 1: Iteration:  600 / 2000 [ 30%]  (Warmup)
Chain 1: Iteration:  800 / 2000 [ 40%]  (Warmup)
Chain 1: Iteration: 1000 / 2000 [ 50%]  (Warmup)
Chain 1: Iteration: 1001 / 2000 [ 50%]  (Sampling)
Chain 1: Iteration: 1200 / 2000 [ 60%]  (Sampling)
Chain 1: Iteration: 1400 / 2000 [ 70%]  (Sampling)
Chain 1: Iteration: 1600 / 2000 [ 80%]  (Sampling)
Chain 1: Iteration: 1800 / 2000 [ 90%]  (Sampling)
Chain 1: Iteration: 2000 / 2000 [100%]  (Sampling)
Chain 1:
Chain 1:  Elapsed Time: 0.225 seconds (Warm-up)
Chain 1:                0.211 seconds (Sampling)
Chain 1:                0.436 seconds (Total)
Chain 1:

SAMPLING FOR MODEL 'bernoulli' NOW (CHAIN 2).
Chain 2:
Chain 2: Gradient evaluation took 1.7e-05 seconds
```

```
Chain 2: 1000 transitions using 10 leapfrog steps per transition would take 0.17 seconds.
Chain 2: Adjust your expectations accordingly!
Chain 2:
Chain 2:
Chain 2: Iteration:    1 / 2000 [  0%]  (Warmup)
Chain 2: Iteration:  200 / 2000 [ 10%]  (Warmup)
Chain 2: Iteration:  400 / 2000 [ 20%]  (Warmup)
Chain 2: Iteration:  600 / 2000 [ 30%]  (Warmup)
Chain 2: Iteration:  800 / 2000 [ 40%]  (Warmup)
Chain 2: Iteration: 1000 / 2000 [ 50%]  (Warmup)
Chain 2: Iteration: 1001 / 2000 [ 50%]  (Sampling)
Chain 2: Iteration: 1200 / 2000 [ 60%]  (Sampling)
Chain 2: Iteration: 1400 / 2000 [ 70%]  (Sampling)
Chain 2: Iteration: 1600 / 2000 [ 80%]  (Sampling)
Chain 2: Iteration: 1800 / 2000 [ 90%]  (Sampling)
Chain 2: Iteration: 2000 / 2000 [100%]  (Sampling)
Chain 2:
Chain 2:  Elapsed Time: 0.208 seconds (Warm-up)
Chain 2:                0.214 seconds (Sampling)
Chain 2:                0.422 seconds (Total)
Chain 2:
```

```
print(out_mim_rd)
```

```
Object of class 'outstandR'
ITC algorithm: MIM
Model: binomial
Scale: risk_difference
Common treatment: C
Individual patient data study: A vs C
Aggregate level data study: B vs C
Confidence interval level: 0.95

Contrasts:

# A tibble: 3 x 5
  Treatments Estimate Std.Error lower.0.95 upper.0.95
  <chr>         <dbl>     <dbl>      <dbl>      <dbl>
1 AB            0.197   0.433        -1.09      1.49
2 AC           -0.144  -0.00223     NaN       NaN
3 BC           -0.341   0.436        -1.63      0.952
```

```
Absolute:

# A tibble: 3 x 5
  Treatments Estimate Std.Error lower.0.95 upper.0.95
  <chr>         <dbl>     <dbl>      <dbl>      <dbl>
1 A             0.299   0.00272      0.197      0.401
2 B             0.102   0.440       -1.20       1.40
3 C             0.444   0.00476      0.308      0.579
```

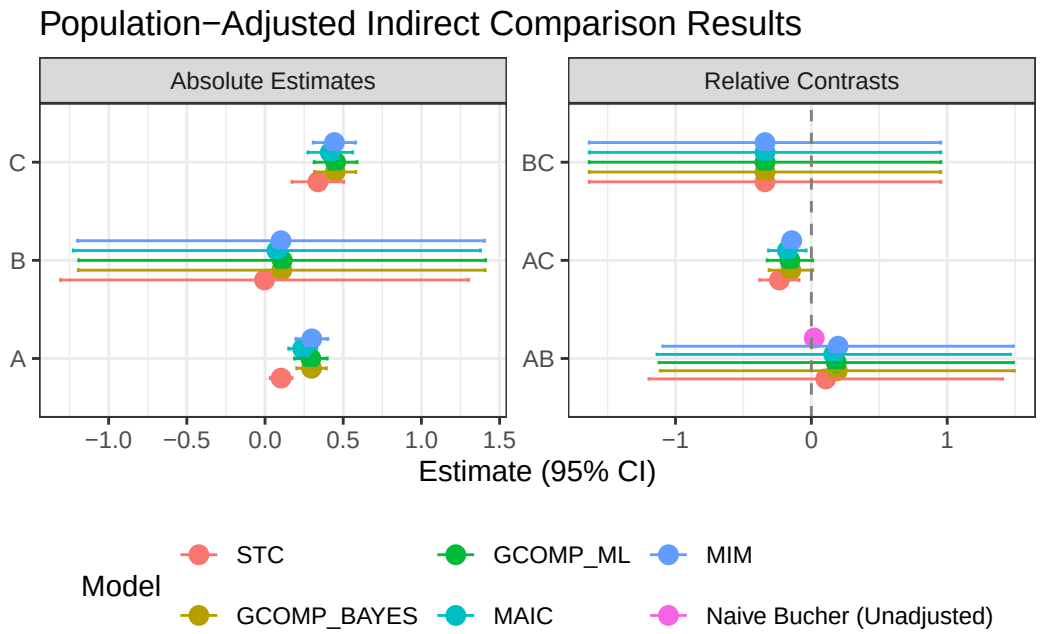
```r
out_stc_rd <- outstandR(
  ipd_trial = AC_IPD_binY_contX,
  ald_trial = BC_ALD_binY_contX,
  strategy = strategy_stc(
    formula = formula_list_bin,
    family = binomial(link = "logit")
  ),
  scale = "risk_difference",
  seed = 12345
)

print(out_stc_rd)
```

```
Object of class 'outstandR'
ITC algorithm: STC
Model: binomial
Scale: risk_difference
Common treatment: C
Individual patient data study: A vs C
Aggregate level data study: B vs C
Confidence interval level: 0.95

Contrasts:

# A tibble: 3 x 5
  Treatments Estimate Std.Error lower.0.95 upper.0.95
  <chr>         <dbl>     <dbl>      <dbl>      <dbl>
1 AB            0.106   0.441       -1.20       1.41
2 AC           -0.235   0.00547     -0.380     -0.0904
3 BC           -0.341   0.436       -1.63       0.952

Absolute:
```

```
# A tibble: 3 x 5
  Treatments Estimate Std.Error lower.0.95 upper.0.95
  <chr>         <dbl>     <dbl>      <dbl>      <dbl>
1 A             0.103   0.00126     0.0331      0.172
2 B            -0.00325 0.443      -1.31        1.30
3 C             0.338   0.00713     0.173       0.504
```

Plot forest plot comparing all results on the new scale.

```
plot(out_stc_rd, out_gcomp_bayes_rd, out_gcomp_ml_rd, out_maic_rd, out_mim_rd)
```



7. Calculate the unadjusted (naive) estimates and plot them against your adjusted outputs. This visualization demonstrates exactly how much the population adjustment reduced bias caused by mismatched effect modifiers.

> **Tip**
>
> **Task:** Calculate the raw probabilities from the IPD and ALD, compute the naive anchored Risk Difference `(A - C) - (B - C)`, bind this with your `out_maic_rd` and `out_gcomp_ml` contrast estimates, and generate a forest plot.

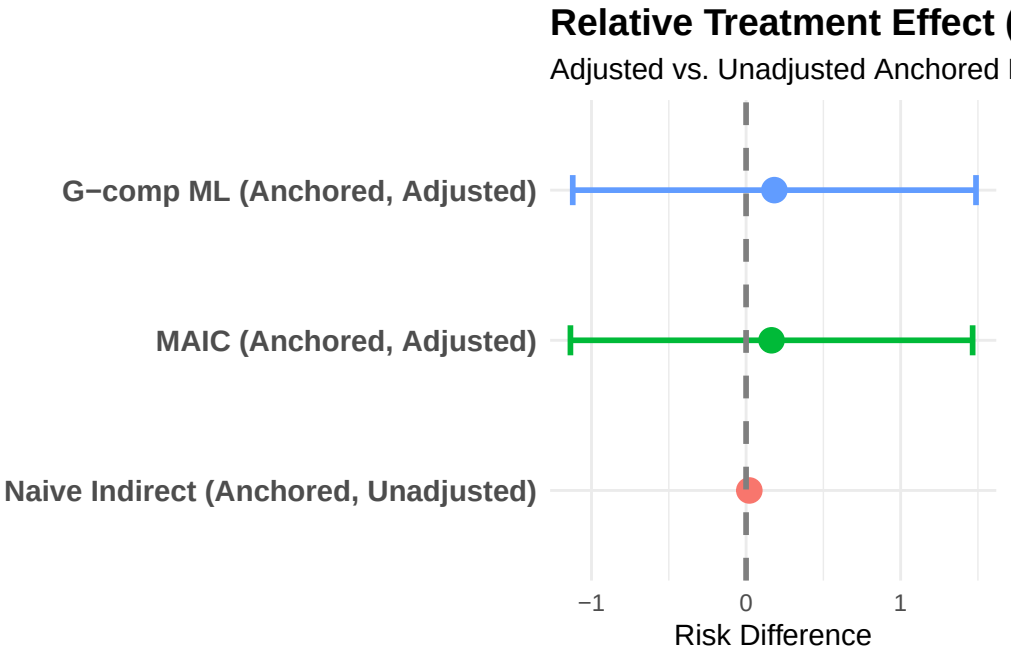
```
# 1. Raw probabilities from IPD
prob_A_raw <- mean(AC_IPD_binY_contX$y[AC_IPD_binY_contX$trt == "A"])
prob_C_raw_AC <- mean(AC_IPD_binY_contX$y[AC_IPD_binY_contX$trt == "C"])
```

```r
# 2. Raw probabilities from ALD
prob_B_raw <- BC_ALD_binY_contX |>
  filter(variable == "y", statistic == "mean", trt == "B") |>
  pull(value)
prob_C_raw_BC <- BC_ALD_binY_contX |>
  filter(variable == "y", statistic == "mean", trt == "C") |>
  pull(value)

# 3. Naive Anchored Risk Difference
# Explicit manual calculation (direct subtraction):
naive_anchored_rd_explicit <- (prob_A_raw - prob_C_raw_AC) - (prob_B_raw - prob_C_raw_BC)

# Or using outstandR's built-in calc_bucher_naive() function:
# This calculates it on the log-odds scale and back-transforms, which is HTA best practice.
naive_results_pkg <- calc_bucher_naive(
  ipd_trial = AC_IPD_binY_contX,
  ald_trial = BC_ALD_binY_contX,
  outcome_model = lin_form_bin,
  family = binomial(link = "logit"),
  ref_trt = "C",
  scale = "risk_difference"
)
naive_anchored_rd <- naive_results_pkg$Estimate

# Note: You can also extract this directly from any of the outstandR output objects
# since the naive Bucher comparison is run automatically under the hood:
# naive_anchored_rd <- out_maic_rd$naive$Estimate

# 4. Extract and bind results
adjusted_results <- data.frame(
  Estimate = c(out_maic_rd$results$contrasts$means$AB,
               out_gcomp_ml_rd$results$contrasts$means$AB),
  lower.0.95 = c(out_maic_rd$results$contrasts$CI$AB[1],
               out_gcomp_ml_rd$results$contrasts$CI$AB[1]),
  upper.0.95 = c(out_maic_rd$results$contrasts$CI$AB[2],
               out_gcomp_ml_rd$results$contrasts$CI$AB[2]),
  Method = c("MAIC (Anchored, Adjusted)", "G-comp ML (Anchored, Adjusted)"))

naive_result <- data.frame(
  Treatments = "AB",
  Estimate = naive_anchored_rd,
  `Std. Error` = NA, lower.0.95 = NA, upper.0.95 = NA,
```

```
  Method = "Naive Indirect (Anchored, Unadjusted)",
  check.names = FALSE
)

all_results_intro <- bind_rows(adjusted_results, naive_result)

# Order the factor for plotting
all_results_intro$Method <- factor(all_results_intro$Method, levels = c(
  "Naive Indirect (Anchored, Unadjusted)",
  "MAIC (Anchored, Adjusted)",
  "G-comp ML (Anchored, Adjusted)"
))
```

5. Build the Forest Plot

```
ggplot(all_results_intro, aes(x = Estimate, y = Method, color = Method)) +
  geom_point(size = 4) +
  geom_errorbarh(aes(xmin = lower.0.95, xmax = upper.0.95), height = 0.2, linewidth = 1, na.
  geom_vline(xintercept = 0, linetype = "dashed", color = "gray50", linewidth = 1) +
  theme_minimal() +
  labs(
    title = "Relative Treatment Effect (A vs B)",
    subtitle = "Adjusted vs. Unadjusted Anchored Estimates",
    x = "Risk Difference",
    y = ""
  ) +
  theme(
    legend.position = "none",
    axis.text.y = element_text(size = 11, face = "bold"),
    plot.title = element_text(face = "bold", size = 14)
  )
```



Advanced Applications: Mixed Data and Custom Distributions

Background

In reality, patient data isn't purely continuous. We often have discrete/binary covariates (e.g., Sex, Prior Treatment Status). By default, the simulation step in G-computation assumes all covariates in the target population follow a normal distribution. For highly skewed data (like costs or age) or binary variables, this assumption is flawed. `{outstandR}` allows you to pass custom distributions so they can be appropriately parameterized using the moments available in your ALD dataset.

Exercises

1. Load the continuous outcome dataset that features mixed covariates (`X1`, `X3` are continuous; `X2`, `X4` are binary), and update the formulas.

```
data(AC_IPD_contY_mixedX)
data(BC_ALD_contY_mixedX)

lin_form_mixed <- as.formula("y ~ X1 + X2 + X3 + trt + trt:(X1 + X2 + X4)")
bal_form_mixed <- as.formula("~ X1 + X2 + X4")

formula_list_mixed <- list(outcome_model = lin_form_mixed, balance_model = bal_form_mixed)
```

2. Run a ML G-computation on the continuous outcome data using custom marginal distributions.

> **Tip**
>
> **Task:** Force covariate `X1` to be simulated as a `gamma` distribution, and `X2` to be simulated as a `binom` (binomial) distribution. (`X3` and `X4` will default back to normal). Pass these as a named vector to the `marginal_distns` argument.

```
custom_distns <- c(X1 = "gamma", X2 = "binom")

out_gcomp_custom <- outstandR(
  ipd_trial = AC_IPD_contY_mixedX,
  ald_trial = BC_ALD_contY_mixedX,
  strategy = strategy_gcomp_ml(
    formula = formula_list_mixed,
    family = gaussian(link = "identity"),
    marginal_distns = custom_distns,
    N = 1000
  ),
  seed = 12345
)

print(out_gcomp_custom)
```

```
Object of class 'outstandR'
ITC algorithm: GCOMP_ML
Model: gaussian
Scale: mean_difference
Common treatment: C
Individual patient data study: A vs C
Aggregate level data study: B vs C
Confidence interval level: 0.95

Contrasts:

# A tibble: 3 x 5
  Treatments Estimate Std.Error lower.0.95 upper.0.95
  <chr>         <dbl>     <dbl>      <dbl>      <dbl>
1 AB           -0.293    0.0930     -0.891      0.305
2 AC           -1.22     0.0619     -1.71      -0.732
3 BC           -0.926    0.0311     -1.27      -0.581
```

```
Absolute:

# A tibble: 3 x 5
  Treatments Estimate Std.Error lower.0.95 upper.0.95
  <chr>         <dbl>     <dbl>      <dbl>      <dbl>
1 A            -0.507    0.0176     -0.767     -0.247
2 B            -0.214    0.0804     -0.769      0.342
3 C             0.713    0.0493      0.277      1.15
```

3. *(Optional)* Bayesian G-computation and Multiple Imputation Marginalization (MIM) provide a highly robust framework for propagating uncertainty natively without bootstrapping. Because these utilize MCMC sampling under the hood (via `rstanarm`), they take slightly longer to run.

```
# Bayesian G-Computation
out_gcomp_bayes <- outstandR(
  ipd_trial = AC_IPD_contY_mixedX,
  ald_trial = BC_ALD_contY_mixedX,
  strategy = strategy_gcomp_bayes(
    formula = formula_list_mixed,
    family = gaussian(link = "identity")
  ),
  seed = 12345
)
```

```
SAMPLING FOR MODEL 'continuous' NOW (CHAIN 1).
Chain 1:
Chain 1: Gradient evaluation took 5.4e-05 seconds
Chain 1: 1000 transitions using 10 leapfrog steps per transition would take 0.54 seconds.
Chain 1: Adjust your expectations accordingly!
Chain 1:
Chain 1:
Chain 1: Iteration:    1 / 2000 [  0%]  (Warmup)
Chain 1: Iteration:  500 / 2000 [ 25%]  (Warmup)
Chain 1: Iteration: 1000 / 2000 [ 50%]  (Warmup)
Chain 1: Iteration: 1001 / 2000 [ 50%]  (Sampling)
Chain 1: Iteration: 1500 / 2000 [ 75%]  (Sampling)
Chain 1: Iteration: 2000 / 2000 [100%]  (Sampling)
Chain 1:
Chain 1:  Elapsed Time: 0.08 seconds (Warm-up)
```

```
Chain 1:                0.086 seconds (Sampling)
Chain 1:                0.166 seconds (Total)
Chain 1:

SAMPLING FOR MODEL 'continuous' NOW (CHAIN 2).
Chain 2:
Chain 2: Gradient evaluation took 1.1e-05 seconds
Chain 2: 1000 transitions using 10 leapfrog steps per transition would take 0.11 seconds.
Chain 2: Adjust your expectations accordingly!
Chain 2:
Chain 2:
Chain 2: Iteration:    1 / 2000 [  0%]  (Warmup)
Chain 2: Iteration:  500 / 2000 [ 25%]  (Warmup)
Chain 2: Iteration: 1000 / 2000 [ 50%]  (Warmup)
Chain 2: Iteration: 1001 / 2000 [ 50%]  (Sampling)
Chain 2: Iteration: 1500 / 2000 [ 75%]  (Sampling)
Chain 2: Iteration: 2000 / 2000 [100%]  (Sampling)
Chain 2:
Chain 2:  Elapsed Time: 0.073 seconds (Warm-up)
Chain 2:                0.09 seconds (Sampling)
Chain 2:                0.163 seconds (Total)
Chain 2:
```

```r
# Multiple Imputation Marginalization (MIM)
out_mim <- outstandR(
  ipd_trial = AC_IPD_contY_mixedX,
  ald_trial = BC_ALD_contY_mixedX,
  strategy = strategy_mim(
    formula = formula_list_mixed,
    family = gaussian(link = "identity")
  ),
  seed = 12345
)
```

```
SAMPLING FOR MODEL 'continuous' NOW (CHAIN 1).
Chain 1:
Chain 1: Gradient evaluation took 2e-05 seconds
Chain 1: 1000 transitions using 10 leapfrog steps per transition would take 0.2 seconds.
Chain 1: Adjust your expectations accordingly!
Chain 1:
Chain 1:
```

```
Chain 1: Iteration:    1 / 2000 [  0%]  (Warmup)
Chain 1: Iteration:  200 / 2000 [ 10%]  (Warmup)
Chain 1: Iteration:  400 / 2000 [ 20%]  (Warmup)
Chain 1: Iteration:  600 / 2000 [ 30%]  (Warmup)
Chain 1: Iteration:  800 / 2000 [ 40%]  (Warmup)
Chain 1: Iteration: 1000 / 2000 [ 50%]  (Warmup)
Chain 1: Iteration: 1001 / 2000 [ 50%]  (Sampling)
Chain 1: Iteration: 1200 / 2000 [ 60%]  (Sampling)
Chain 1: Iteration: 1400 / 2000 [ 70%]  (Sampling)
Chain 1: Iteration: 1600 / 2000 [ 80%]  (Sampling)
Chain 1: Iteration: 1800 / 2000 [ 90%]  (Sampling)
Chain 1: Iteration: 2000 / 2000 [100%]  (Sampling)
Chain 1:
Chain 1:  Elapsed Time: 0.081 seconds (Warm-up)
Chain 1:                0.085 seconds (Sampling)
Chain 1:                0.166 seconds (Total)
Chain 1:

SAMPLING FOR MODEL 'continuous' NOW (CHAIN 2).
Chain 2:
Chain 2: Gradient evaluation took 1.1e-05 seconds
Chain 2: 1000 transitions using 10 leapfrog steps per transition would take 0.11 seconds.
Chain 2: Adjust your expectations accordingly!
Chain 2:
Chain 2:
Chain 2: Iteration:    1 / 2000 [  0%]  (Warmup)
Chain 2: Iteration:  200 / 2000 [ 10%]  (Warmup)
Chain 2: Iteration:  400 / 2000 [ 20%]  (Warmup)
Chain 2: Iteration:  600 / 2000 [ 30%]  (Warmup)
Chain 2: Iteration:  800 / 2000 [ 40%]  (Warmup)
Chain 2: Iteration: 1000 / 2000 [ 50%]  (Warmup)
Chain 2: Iteration: 1001 / 2000 [ 50%]  (Sampling)
Chain 2: Iteration: 1200 / 2000 [ 60%]  (Sampling)
Chain 2: Iteration: 1400 / 2000 [ 70%]  (Sampling)
Chain 2: Iteration: 1600 / 2000 [ 80%]  (Sampling)
Chain 2: Iteration: 1800 / 2000 [ 90%]  (Sampling)
Chain 2: Iteration: 2000 / 2000 [100%]  (Sampling)
Chain 2:
Chain 2:  Elapsed Time: 0.091 seconds (Warm-up)
Chain 2:                0.098 seconds (Sampling)
Chain 2:                0.189 seconds (Total)
Chain 2:
```

Model Diagnostics, Misspecification & Variable Selection

Background

A critical vulnerability of regression-based population adjustment is model misspecification. If our outcome model fails to capture the relationship between the covariates, treatment, and outcome (e.g., omitting the effect modifiers as interaction terms), our standardized estimates will be biased.

Conversely, in weighting methods like MAIC, the temptation is often to throw every available baseline characteristic into the matching model. Balancing on variables that are purely prognostic (or adding highly correlated noise) can drastically reduce your Effective Sample Size (ESS) without necessarily reducing bias.

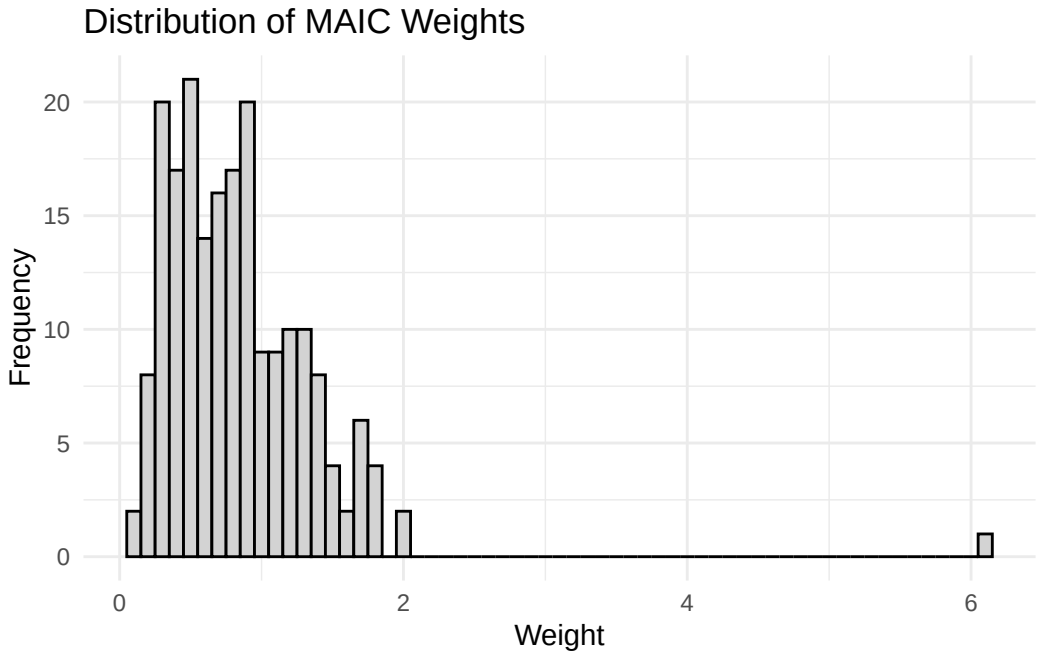
Exercises

1. Dive into the `outstandR` object to assess model fit by checking the Effective Sample Size (ESS) and plotting the distribution of weights from your MAIC analysis.

```
cat("MAIC Effective Sample Size:", out_maic$model$ESS, "\n")
```

```
MAIC Effective Sample Size: 136.8594
```

```
ggplot(data.frame(weights = out_maic$model$weights), aes(x = weights)) +
  geom_histogram(binwidth = 0.1, color = "black", fill = "lightgray") +
  labs(title = "Distribution of MAIC Weights", x = "Weight", y = "Frequency") +
  theme_minimal()
```



If a large percentage of your weights are near zero, your MAIC estimates may be highly unstable, suggesting poor covariate overlap between your IPD and target population.

2. Fit a deliberately misspecified G-computation model to observe the consequences. We will purposefully drop the `trt:EM` interaction terms.

```
bad_form_bin <- as.formula("y ~ PF_cont_1 + PF_cont_2 + EM_cont_1 + EM_cont_2 + trt")

out_gcomp_bad <- outstandR(
  ipd_trial = AC_IPD_binY_contX,
  ald_trial = BC_ALD_binY_contX,
  strategy = strategy_gcomp_ml(
    formula = list(outcome_model = bad_form_bin),
    family = binomial(link = "logit"),
    N = 1000
  ),
  scale = "risk_difference",
  seed = 12345
)

cat("Misspecified Risk Difference (A vs B):", out_gcomp_bad$results$contrasts$means$AB, "\n")
```

```
Misspecified Risk Difference (A vs B): 0.05618592
```

```r
cat("Correct ML G-comp Risk Difference (A vs B):", out_gcomp_ml_rd$results$contrasts$means$AB
```

```
Correct ML G-comp Risk Difference (A vs B): 0.1830297
```

3. Next, let's explore the Bias-Variance trade-off by running a minimal, correctly specified MAIC balancing *only one* effect modifier.

```r
bal_form_minimal <- as.formula("~ EM_cont_1")

out_maic_minimal <- outstandR(
  ipd_trial = AC_IPD_binY_contX,
  ald_trial = BC_ALD_binY_contX,
  strategy = strategy_maic(
    formula = list(
      outcome_model = formula("y ~ trt"),
      balance_model = bal_form_minimal
    ),
    family = binomial(link = "logit")
  ),
  seed = 12345
)

cat("Minimal Model ESS:", out_maic_minimal$model$ESS, "\n")
```

```
Minimal Model ESS: 146.9627
```

```r
cat("Minimal Model Std. Error (A vs B):", out_maic_minimal$results$contrasts$variances$AB, "
```

```
Minimal Model Std. Error (A vs B): 0.2497449
```

4. Now, run a "Kitchen Sink" MAIC.

> **Tip**
>
> **Task:** Create a new balance formula (`bal_form_maximal`) that includes all four continuous covariates. Run the MAIC again and observe how the inclusion of purely prognostic factors impacts the ESS and the Standard Errors.

```r
bal_form_maximal <- as.formula("~ PF_cont_1 + PF_cont_2 + EM_cont_1 + EM_cont_2")

out_maic_maximal <- outstandR(
  ipd_trial = AC_IPD_binY_contX,
  ald_trial = BC_ALD_binY_contX,
  strategy = strategy_maic(
    formula = list(
      outcome_model = formula("y ~ trt"),
      balance_model = bal_form_maximal
    ),
    family = binomial(link = "logit")
  ),
  seed = 12345
)

cat("Maximal Model ESS:", out_maic_maximal$model$ESS, "\n")
```

```
Maximal Model ESS: 112.1504
```

```r
cat("Maximal Model Std. Error (A vs B):", out_maic_maximal$results$contrasts$variances$AB, "
```

```
Maximal Model Std. Error (A vs B): 0.2666855
```

Unanchored Comparisons and Visualization

Background

So far, we have performed **anchored** comparisons, relying on Treatment C as a common comparator bridge. If the AC trial was actually a single-arm trial of Treatment A, we no longer assume that unmeasured prognostic factors cancel out. Our adjustment must account for **all** prognostic factors and effect modifiers.

Because `{outstandR}` is designed to compute pairwise contrasts against a reference treatment, passing a single-arm dataset directly will break the internal contrast factory. However, we can use a **"Dummy Anchor" approach**. We will leave the full AC and BC datasets intact but fit a fully specified model (all PFs and EMs). Then, we will ignore the relative contrasts and only extract the absolute marginalized estimate for Treatment A to simulate an unanchored comparison against the raw Treatment B data.

Exercises

1. Run the G-computation including ALL covariates. This satisfies the assumption of absolute exchangeability required for unanchored comparisons.

```
full_form <- as.formula("y ~ PF_cont_1 + PF_cont_2 + EM_cont_1 + EM_cont_2 + trt")

out_gcomp_unanchored <- outstandR(
  ipd_trial = AC_IPD_binY_contX,
  ald_trial = BC_ALD_binY_contX,
  strategy = strategy_gcomp_ml(
    formula = list(outcome_model = full_form),
    family = binomial(link = "logit"),
    N = 1000
  ),
  scale = "risk_difference",
  seed = 12345
)
```

2. Extract the adjusted absolute effect for Treatment A, and compare it directly to the raw, unadjusted data for Treatment B.

> **Tip**
>
> **Task:** Extract the `results$` for Treatment A from the `absolute_effects` table inside your `out_gcomp_unanchored` object. Subtract the raw observed mean for Treatment B (from the ALD dataset) to calculate the unanchored Risk Difference.

```
# 1. Extract the marginalized probability for A in the target population
prob_A_unanchored <- out_gcomp_unanchored$results$absolute$means$A

# 2. Extract raw observed probability for B (from the ALD)
prob_B_raw <- BC_ALD_binY_contX |>
  filter(variable == "y", statistic == "mean", trt == "B") |>
  pull(value)

# 3. Calculate the unanchored Risk Difference
unanchored_rd <- prob_A_unanchored - prob_B_raw

cat("Unanchored Risk Difference (A vs B):", round(unanchored_rd, 3), "\n")
```

```
Unanchored Risk Difference (A vs B): -0.035
```

3. Calculate the unadjusted (naive) estimates for comparison.

To see exactly how much our PAIC methods moved the needle, we will calculate the naive estimators. Let p_A and $p_{C(AC)}$ be the raw probabilities of the event in the IPD trial, and let p_B and $p_{C(BC)}$ be the raw probabilities in the ALD trial.

- **Naive Anchored Risk Difference:** This assumes the relative effect is transportable without adjustment (the classic Bucher indirect comparison).

$$\hat{\Delta}_{AB}^{\text{naive\_anchored}} = (p_A - p_{C(AC)}) - (p_B - p_{C(BC)})$$

- **Naive Unanchored Risk Difference:** This drops the common comparator entirely and assumes absolute exchangeability between the two isolated active arms.

$$\hat{\Delta}_{AB}^{\text{naive\_unanchored}} = p_A - p_B$$

> **Warning**
>
> **A Note on Naive Risk Differences:** While mathematically intuitive, standard HTA practice warns against performing a Bucher comparison directly on probabilities. Because probabilities are bounded between 0 and 1, assuming absolute risk differences are strictly additive across different populations can lead to impossible values. Typically, indirect comparisons are performed on relative unconstrained scales (like Log-Odds) and then mapped back to absolute scales.

To adhere to best practices, we will calculate the naive anchored comparison on the Log-Odds scale, and then back-transform it to the probability scale by applying it to the observed baseline odds of Treatment B.

```
# 1. Raw probabilities from IPD (AC trial)
prob_A_raw <- mean(AC_IPD_binY_contX$y[AC_IPD_binY_contX$trt == "A"])
prob_C_raw_AC <- mean(AC_IPD_binY_contX$y[AC_IPD_binY_contX$trt == "C"])

# Convert to Log-Odds and calculate Naive AC Log-OR
lo_A <- log(prob_A_raw / (1 - prob_A_raw))
lo_C_AC <- log(prob_C_raw_AC / (1 - prob_C_raw_AC))
naive_logOR_AC <- lo_A - lo_C_AC

# 2. Raw probabilities from ALD (BC trial)
prob_B_raw <- BC_ALD_binY_contX |>
  filter(variable == "y", statistic == "mean", trt == "B") |> pull(value)
prob_C_raw_BC <- BC_ALD_binY_contX |>
```

```r
  filter(variable == "y", statistic == "mean", trt == "C") |> pull(value)

# Convert to Log-Odds and calculate Naive BC Log-OR
lo_B <- log(prob_B_raw / (1 - prob_B_raw))
lo_C_BC <- log(prob_C_raw_BC / (1 - prob_C_raw_BC))
naive_logOR_BC <- lo_B - lo_C_BC

# 3. Naive Anchored Indirect Log-OR (A vs B)
# Bucher method: (A vs C) - (B vs C)
naive_anchored_logOR <- naive_logOR_AC - naive_logOR_BC

# 4. Back-transform Naive Anchored Log-OR to Risk Difference
# We apply the indirect Log-OR to the baseline odds of Treatment B
odds_B <- exp(lo_B)
odds_A_naive <- odds_B * exp(naive_anchored_logOR)
prob_A_naive <- odds_A_naive / (1 + odds_A_naive)

naive_anchored_rd <- prob_A_naive - prob_B_raw

# Alternatively, the above back-transformation can be done in one line using calc_bucher_naiv
# naive_anchored_rd <- calc_bucher_naive(
#   ipd_trial = AC_IPD_binY_contX,
#   ald_trial = BC_ALD_binY_contX,
#   outcome_model = lin_form_bin,
#   family = binomial(link = "logit"),
#   ref_trt = "C",
#   scale = "risk_difference"
# )$Estimate

# 5. Naive Unanchored Risk Difference
# A simple unadjusted direct comparison of A vs B
naive_unanchored_rd <- prob_A_raw - prob_B_raw

cat("Naive Anchored RD (Back-transformed):", round(naive_anchored_rd, 3), "\n")
```

```
Naive Anchored RD (Back-transformed): 0.021
```

```r
cat("Naive Unanchored RD (Direct Subtraction):", round(naive_unanchored_rd, 3), "\n")
```

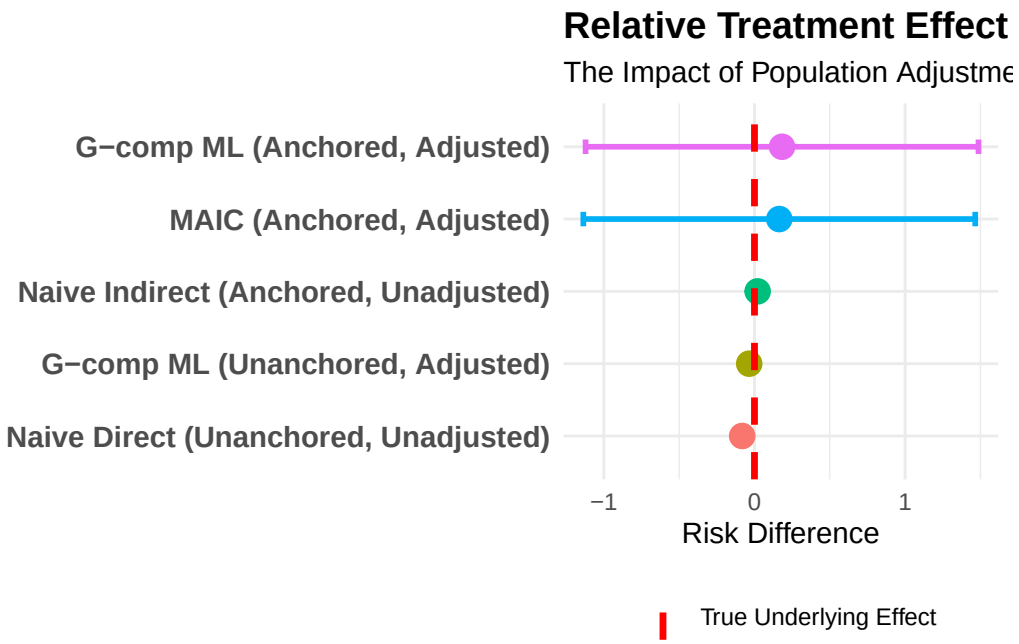
```
Naive Unanchored RD (Direct Subtraction): -0.081
```

4. Extract the previous adjusted results, bind them with the newly computed unanchored estimates, and build the final custom forest plot.

> **Tip**
>
> **The "Ground Truth":** Because this is a synthetic dataset used for teaching, we actually know the "true" underlying treatment effect. In this simulation, Treatments A and B were generated using the exact same underlying parameters. Therefore, the **True Risk Difference between A and B is exactly 0.0**. We can use this to see which method successfully removes the bias caused by the mismatched populations.

```
adjusted_results <- data.frame(
  Estimate = c(out_maic_rd$results$contrasts$means$AB,
               out_gcomp_ml_rd$results$contrasts$means$AB),
  lower.0.95 = c(out_maic_rd$results$contrasts$CI$AB[1],
               out_gcomp_ml_rd$results$contrasts$CI$AB[1]),
  upper.0.95 = c(out_maic_rd$results$contrasts$CI$AB[2],
               out_gcomp_ml_rd$results$contrasts$CI$AB[2]),
  Method = c("MAIC (Anchored, Adjusted)", "G-comp ML (Anchored, Adjusted)"))

manual_results <- data.frame(
  Treatments = rep("AB", 3),
  Estimate = c(unanchored_rd, naive_anchored_rd, naive_unanchored_rd),
  `Std. Error` = NA,
  lower.0.95 = NA,
  upper.0.95 = NA,
  Method = c("G-comp ML (Unanchored, Adjusted)",
             "Naive Indirect (Anchored, Unadjusted)",
             "Naive Direct (Unanchored, Unadjusted)"),
  check.names = FALSE
)

all_results_final <- bind_rows(adjusted_results, manual_results)

all_results_final$Method <- factor(all_results_final$Method, levels = c(
  "Naive Direct (Unanchored, Unadjusted)",
  "G-comp ML (Unanchored, Adjusted)",
  "Naive Indirect (Anchored, Unadjusted)",
  "MAIC (Anchored, Adjusted)",
  "G-comp ML (Anchored, Adjusted)"
))
```

```r
ggplot(all_results_final, aes(x = Estimate, y = Method, color = Method)) +
  geom_point(size = 4) +
  geom_errorbarh(aes(xmin = lower.0.95, xmax = upper.0.95), height = 0.2, linewidth = 1, na.r

  # Highlight True Effect (0.0) with a distinct line
  geom_vline(aes(xintercept = 0, linetype = "True Underlying Effect"), color = "red", linewi
  scale_linetype_manual(name = "", values = "dashed") +

  theme_minimal() +
  labs(
    title = "Relative Treatment Effect (A vs B)",
    subtitle = "The Impact of Population Adjustment and Anchoring Assumptions",
    x = "Risk Difference",
    y = ""
  ) +
  theme(
    legend.position = "bottom", # Show the legend so can see the "True Effect" label
    axis.text.y = element_text(size = 11, face = "bold"),
    plot.title = element_text(face = "bold", size = 14)
  ) +
  guides(color = "none") # Hide redundant colour legend for methods
```



> **Note**
>
> Compare the **Adjusted Anchored** vs. the **Adjusted Unanchored** estimates. The difference here represents the heavy lifting the common comparator (Treatment C) was doing to balance out unmeasured prognostic factors.